

Algorithmen und Datenstrukturen

Übung 4: Suchen

Robert

2026-05-29

Wiederholung Binärer Suchbaum

Binärer Suchbaum

- Bäume speichern Daten strukturiert
- ein binärer Suchbaum besteht aus einer Menge von Knoten:

$$v = v_1, v_2, \dots, v_n$$

- jeder Knoten v ist mit einem Element aus der Menge der Schlüssel $S = \{x_1, x_2, \dots, x_n\}$ beschriftet:

$$key : v \rightarrow S$$

Beschriftung

- Bei der Beschriftung der Knoten v wird die Ordnung der Schlüssel S bewahrt (wenn die Schlüssel vorher geordnet waren, sind sie auch im Baum geordnet)
 - der Schlüssel eines Knoten v_i aus dem linken Teilbaum hat einen kleineren Wert als der Schlüssel der Wurzel v_k
 - der Schlüssel eines Knoten v_j aus dem rechten Teilbaum hat einen größeren Wert als der Schlüssel der Wurzel v_k

$$key(v_i) \leq key(v_k) \leq key(v_j)$$

Aufgabe 1

1 a)

Wie sieht ein binärer Suchbaum mit geringstmöglicher Tiefe für die folgenden Zahlen aus?

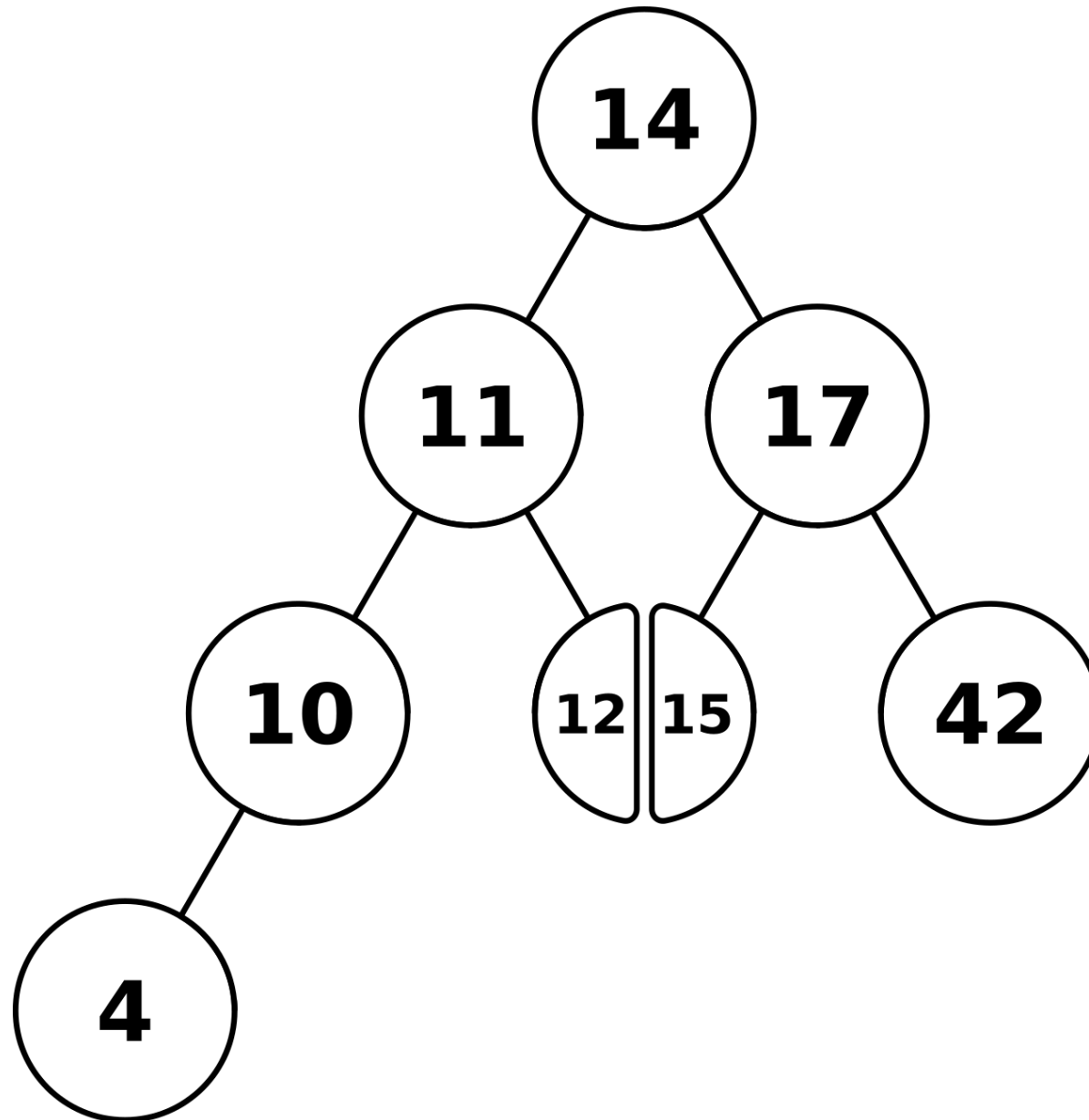
42, 17, 12, 15, 4, 11, 10, 14

1. Die Werte werden der Größe nach geordnet:

4, 10, 11, 12, 14, 15, 17, 42

2. Der Wert in der Mitte bildet die Wurzel (14) und die linke (4, 10, 11, 12) und rechte Hälfte (15, 17, 42) bilden den linken und rechten Teilbaum
3. Bei den Teilmengen wird genauso vorgegangen wie bei der großen Menge:
 - der Wert in der Mitte der Teilmenge bildet die nächste Wurzel
 - die linke Hälfte bildet den linken Teilbaum
 - die rechte Hälfte bildet den rechten Teilbaum

Ergebnis:



1 b)

Wie kann ein neues Element in den Binärbaum eingefügt werden?

- um ein neues Element einzufügen, muss zuerst das Blatt gefunden werden, dessen Wert dem des neuen Elements am nächsten ist
- dann wird diesem Knoten das neue Element als Blatt hinzugefügt

Pseudocode:

```
1 Insert(element, Baum):  
2     if (element < Baum.key):  
3         if (Baum.left):  
4             Insert(element, Baum.left)  
5         else:  
6             Baum.left = new Node(element)  
7     else:  
8         if (Baum.right):  
9             Insert(element, Baum.right)  
10        else:  
11            Baum.right = new Node(element)
```

In welcher Komplexitätsklasse liegt das Verfahren?

- die Suche in einem Binärbaum, der balanciert ist, hat eine Komplexität von $O(\log(n))$ (siehe Übungsfolien 2, Aufgabe 1 aa)).
- wenn der Baum nicht balanciert ist (eine Ebene muss nicht erst komplett voll sein, bevor die nächste angefangen werden kann), dann liegt die Komplexität bei $O(n)$

1 c)

Was muss beim Einfügen eines Elements in einen balancierten Baum beachtet werden, damit die Suche danach immer noch eine Komplexität von $O(\log(n))$ hat?

- nachdem ein Element eingefügt wurde, muss der Baum rebalanciert werden
 - nur bei einem balancierten Baum hat die Suche eine Komplexität von $O(\log(n))$

Rebalancierung

- bei der Rebalancierung gibt es zwei Arten der Rotation:

1. Einfachrotation

- Linksrotation (die Wurzel wird nach links rotiert)
- Rechtsrotation (die Wurzel wird nach rechts rotiert)

2. Doppelrotation

- Links-rechts-Rotation (zuerst wird nach links und dann nach rechts rotiert)
- Rechts-links-Rotation (zuerst wird nach rechts und dann nach links rotiert)

1 d)

Welche Vorteile bietet es, Daten als Baum darzustellen, im Gegensatz zu Darstellung durch sortierte, einfach verkettete Listen oder Arrays, vor allem bezogen auf das Einfügen und Löschen von Elementen?

- wenn Daten in einem balancierten Baum dargestellt sind, haben das Einfügen und Löschen von Elementen jeweils eine Komplexität von $O(\log(n))$
- das Einfügen und Löschen von Elementen bei Listen und Arrays haben hingegen eine Komplexität von $O(n)$

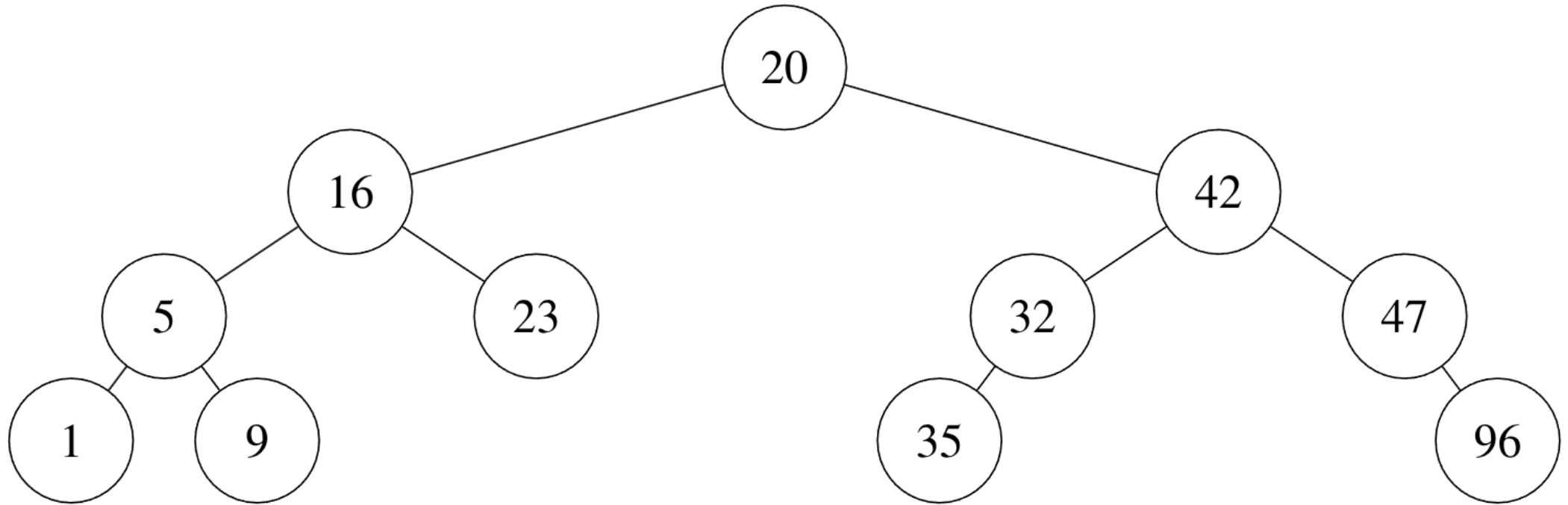
1 e)

Vorteile der Baumdarstellung bezogen auf das Suchen und den Zugriff von Elementen:

- Zugriff:
 - Array: $O(1)$ (mit gegebenem Schlüssel)
 - Baum: $O(\log(n))$
 - Liste: $O(n)$
- Suche
 - Array: $O(\log(n))$ (wenn das Array geordnet ist)
 - Baum: $O(\log(n))$
 - Liste: $O(n)$

Aufgabe 2

Folgender Baum ist gegeben:



2 a)

Handelt es sich um einen binären Suchbaum? Warum?

- nein
 - der Wert 23 ist größer als der Wert der Wurzel und trotzdem liegt er im linken Teilbaum
 - der Wert 35 ist im linken Teilbaum des Knotens 32

2 b)

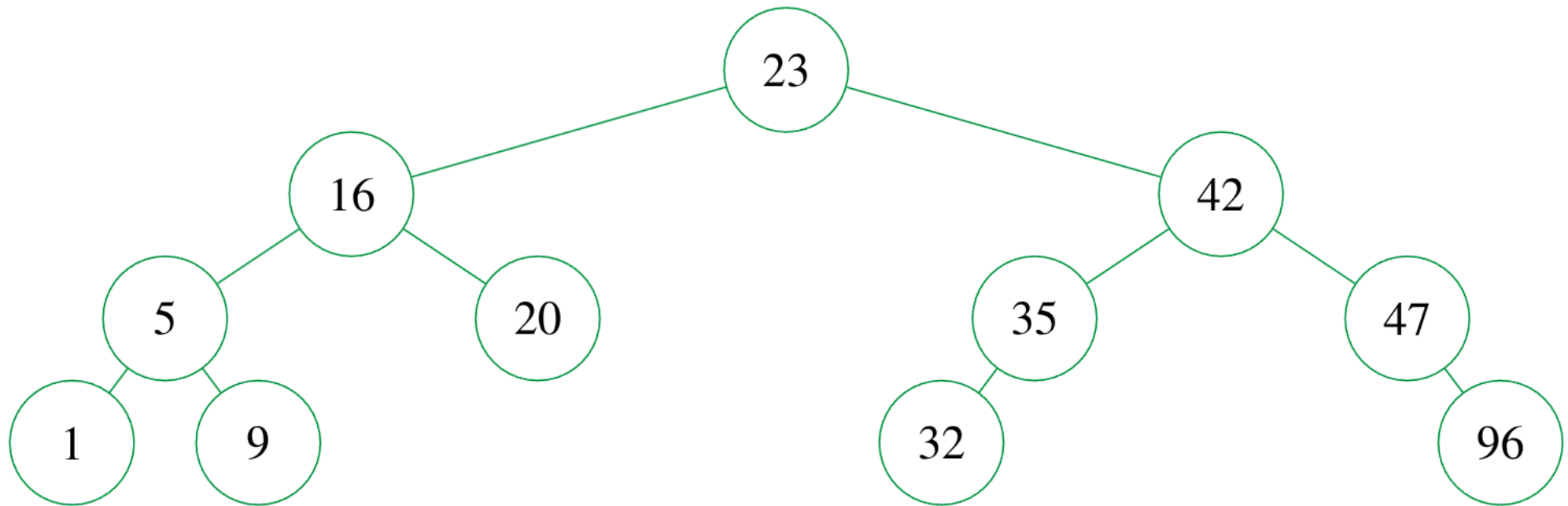
```
1 def get(self, key): # self bezieht sich auf das aktuelle Objekt der Klasse
2     if key == self.key:
3         return self.value
4
5     elif key < self.key and self.left != None:
6         return self.left.get(key)
7
8     elif key > self.key and self.right != None:
9         return self.right.get(key)
10
11     else:
12         return None
```

Welches Ergebnis liefert der `get(self, key)`-Algorithmus bei folgenden Elementen? Welche Knoten wurden besucht?

- `Node(20).get(42)`
 - Ergebnis: 42
 - besuchte Knoten: 20, 42
- `Node(20).get(1)`
 - Ergebnis: 1
 - besuchte Knoten: 20, 16, 5, 1
- `Node(20).get(23)`
 - Ergebnis: `None`
 - besuchte Knoten: 20, 42, 32, 35

2 c)

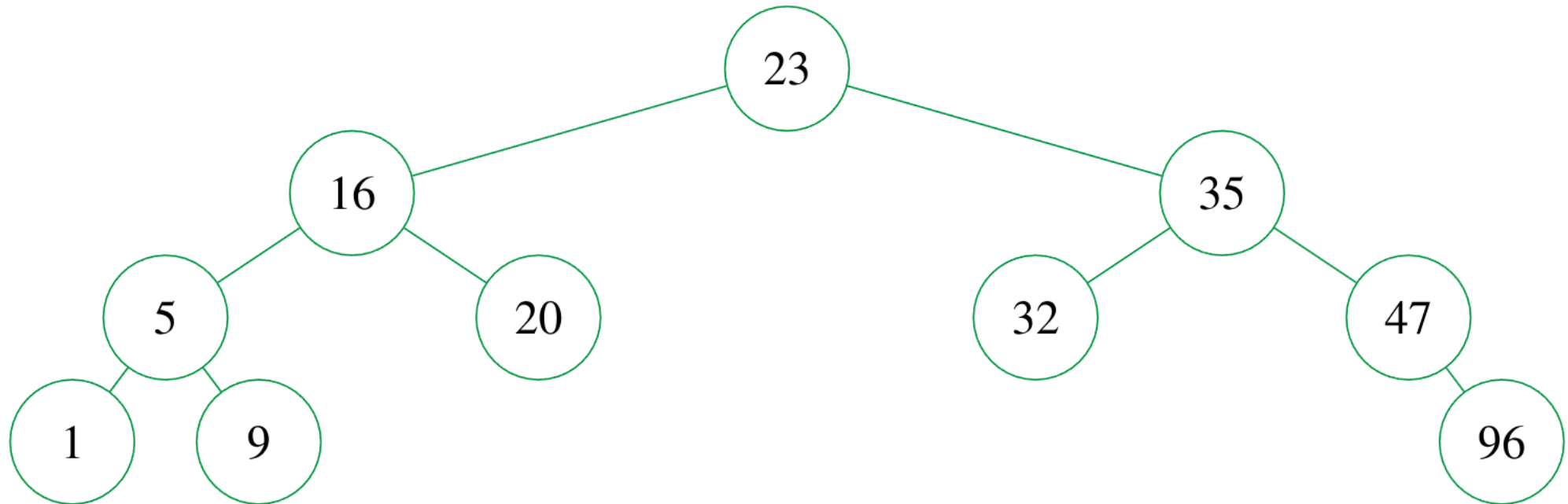
Wie sieht ein Suchbaum aus, der für alle Anfragen der vorherigen Aufgabe ein Ergebnis liefert?



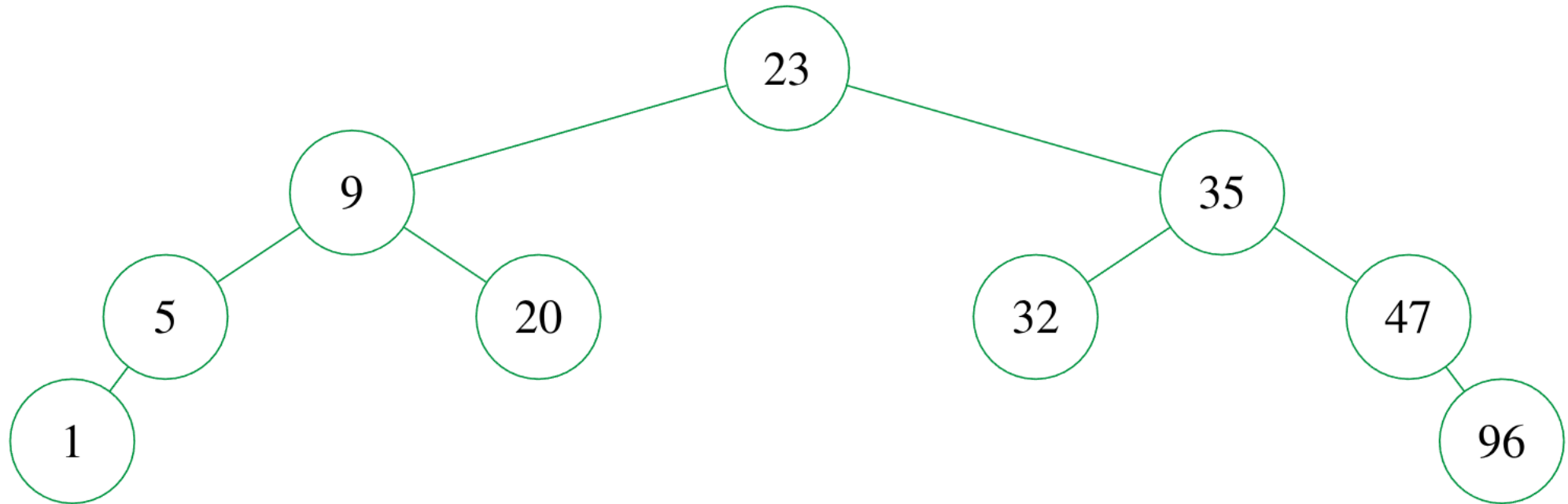
2 d)

Wie sieht der Baum nach den folgenden Operationen aus?

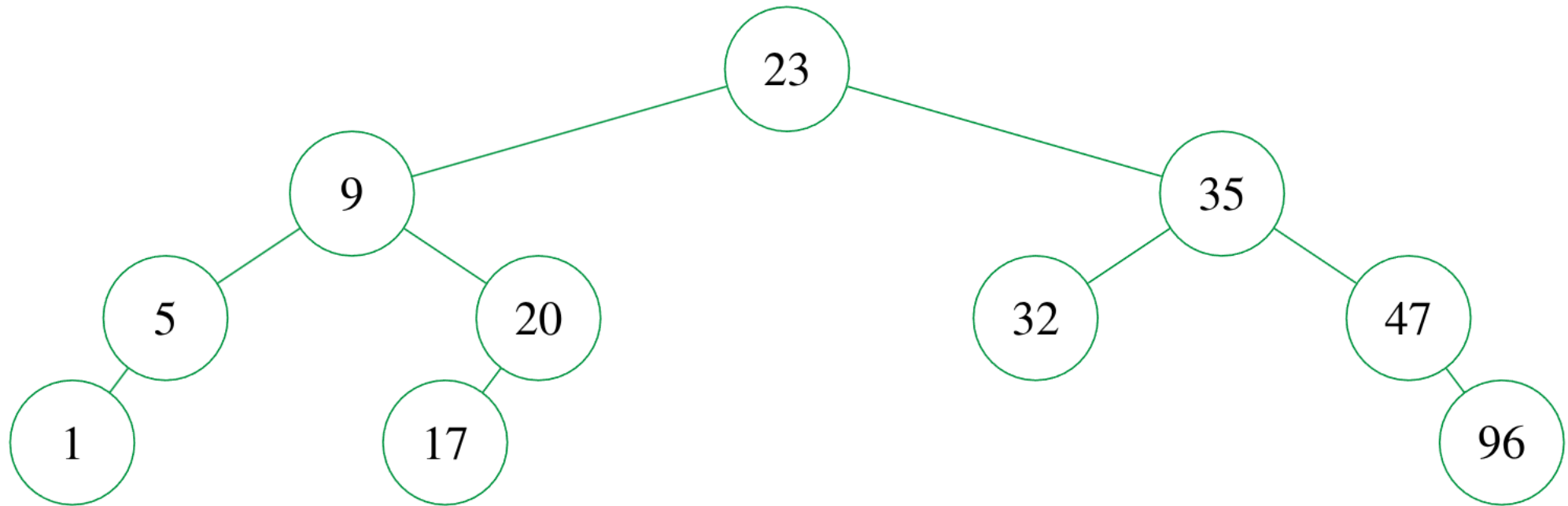
- `remove(42)`



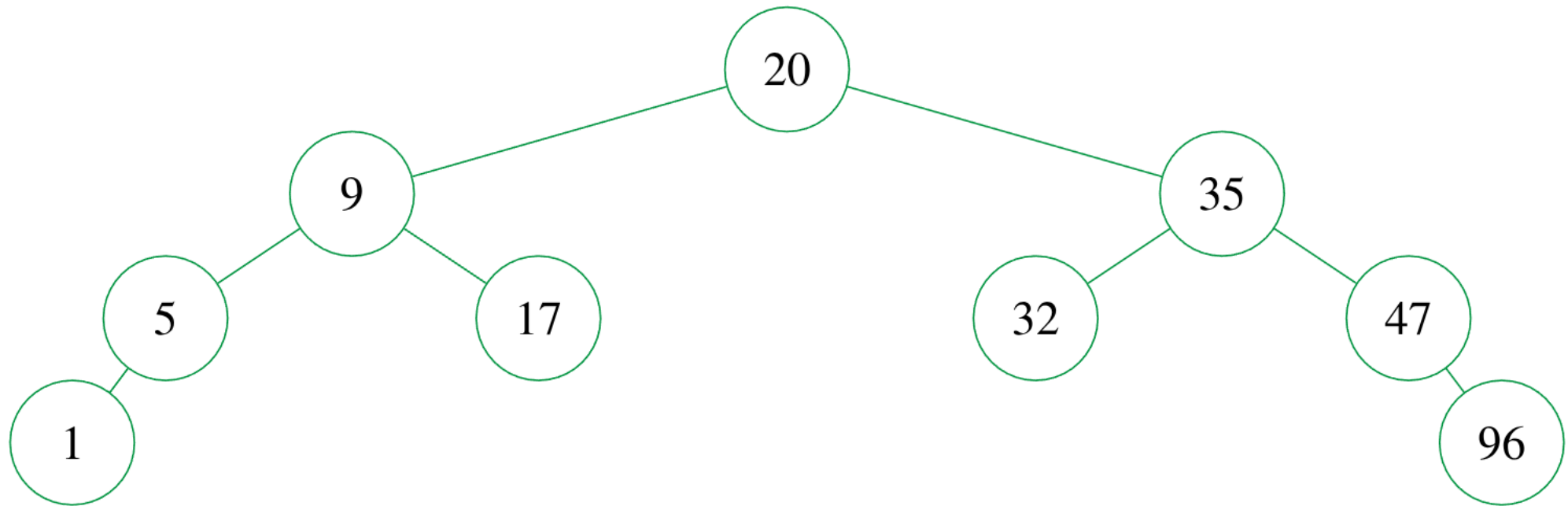
- `remove(16)`



- `put(17, 17)`



- `remove(23)`



Wiederholung Splay Tree

Splay Trees

- Splay-Trees sind dynamische, optimale Suchbäume, die sich dynamisch an die Anfrageverteilung anpassen
 - statische, optimale Suchbäume:
 - die Verteilung der Anfragen ist vorher bekannt und kann bei der Konstruktion des Suchbaums berücksichtigt werden
 - dynamische, optimale Suchbäume:
 - die Verteilung der Anfragen ist nicht vorher bekannt; die Bäume passen sich dynamisch an

- bei beiden gilt, dass die Ordnung der Elemente beibehalten wird
- bei Splay-Trees wird der angefragte Schlüssel durch Rotationen zur neuen Wurzel des Baums → Operation Splay
 - das sorgt dafür, dass oft angefragte Knoten weiter oben im Baum zu finden sind und deren Zugriff somit weniger Zeit kostet
 - die Funktionen Suchen und Einfügen sind mit der Operation Splay gekoppelt

Operation Splay

- die Operation Splay repositioniert das angefragte Element zur Wurzel des Baums
- es gibt zwei Varianten:

1. Bottom-Up Splay

2. Top-Down Splay

1. Bottum-Up Splay:

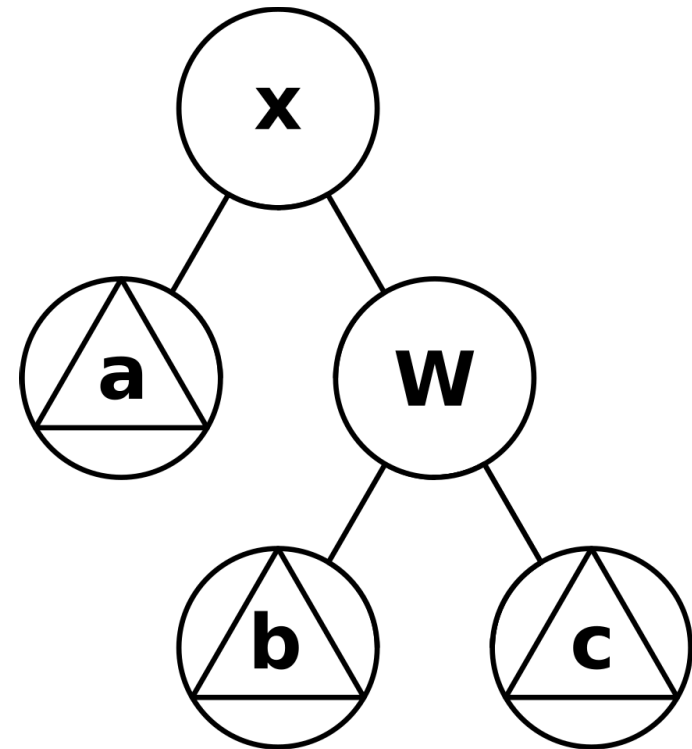
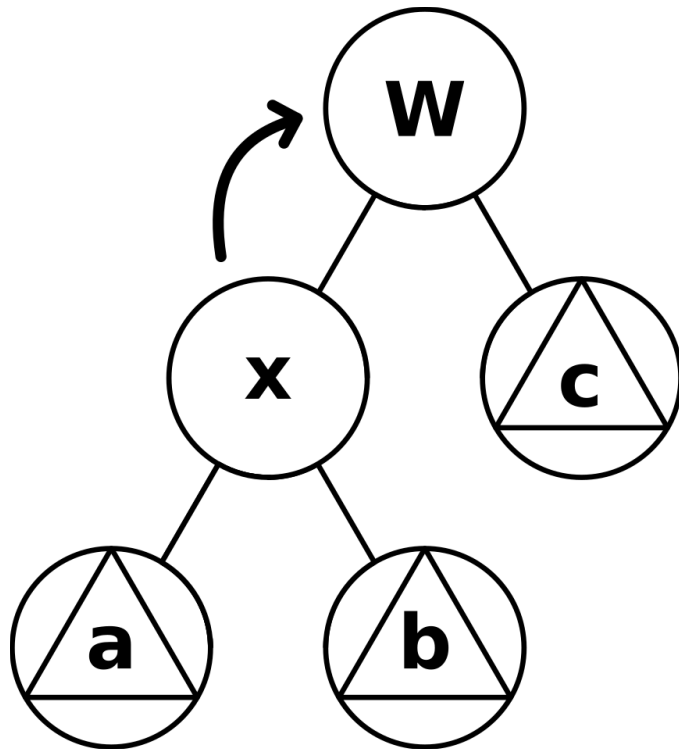
- Suchphase: Bei der Suche wird von der Wurzel zum gesuchten Knoten abgestiegen
- Splayphase: Der Knoten wird nach oben rotiert

2. Top-Down Splay:

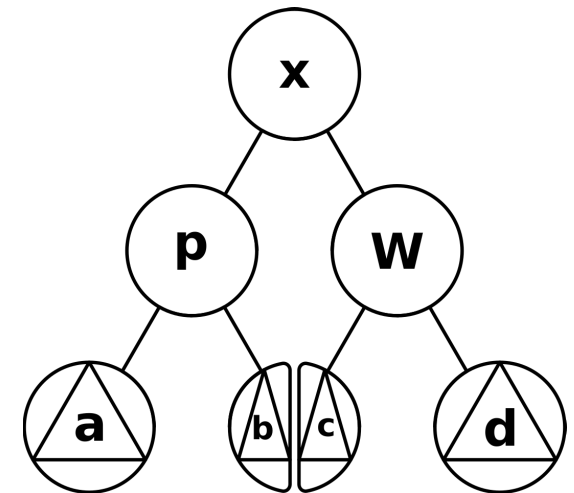
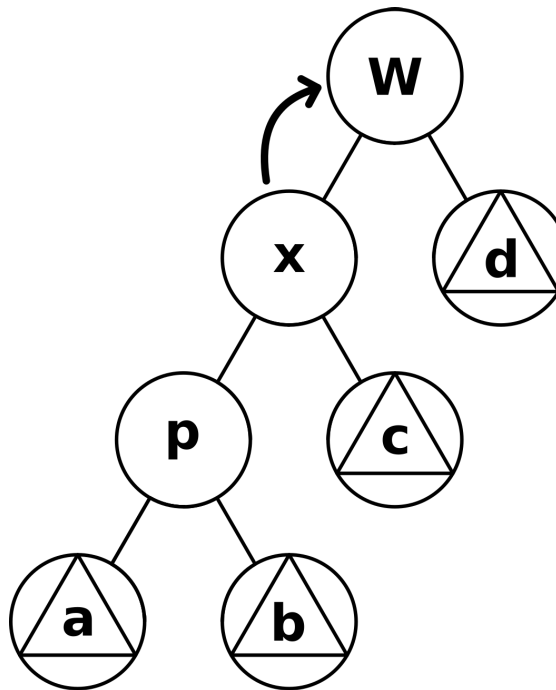
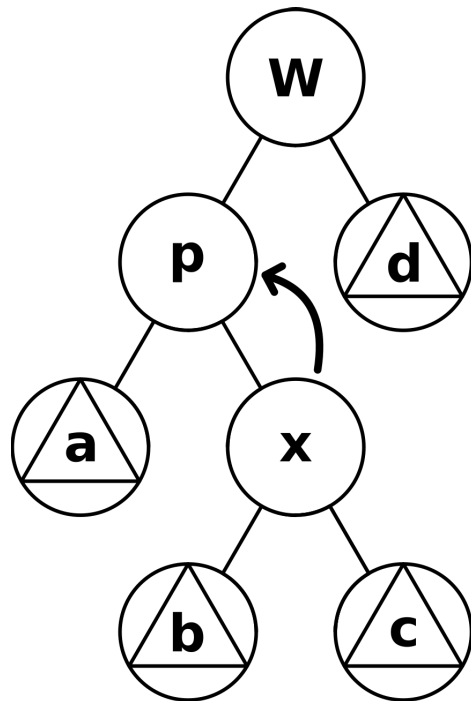
- Alle Knoten, die auf dem Pfad zum gesuchten Knoten liegen, werden nur einmal behandelt
 - anstatt den Pfad hinabzusteigen, werden die Knoten von der Wurzel aus nach oben gezogen (rotiert)
 - kleinere Knoten werden nach links und größere nach rechts rotiert

Beispiel Bottom-Up Splay

- wenn der gesuchte Knoten x die Wurzel W als Vorgänger hat, wird eine einfache Rechts- bzw. Linksrotation durchgeführt (zig bzw. zag):

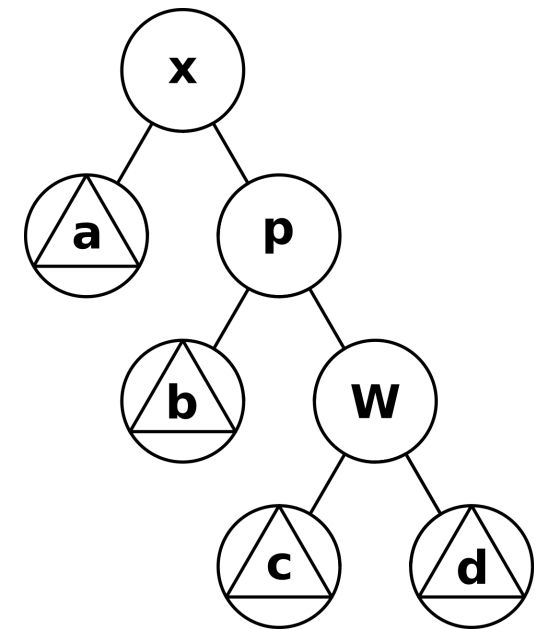
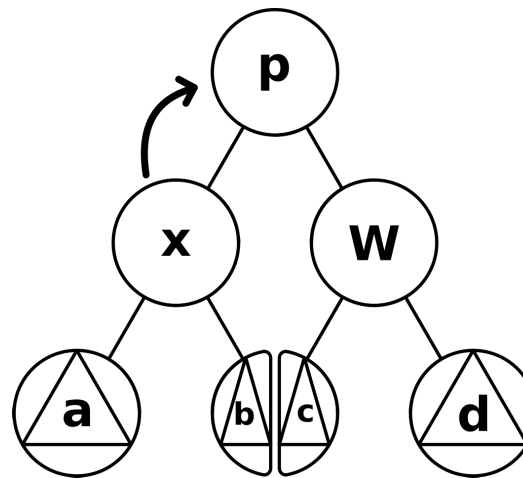
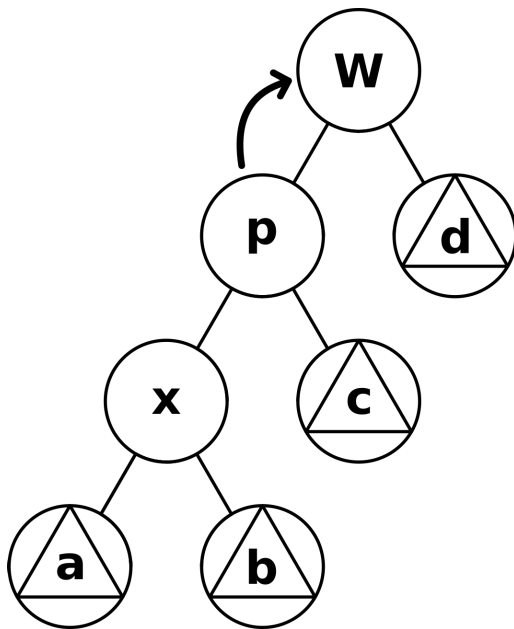


- wenn der gesuchte Knoten x ein rechter Unterbaum seines Vorgängers p und sein Vorgänger p ein linker Unterbaum der Wurzel W ist wird eine entsprechende LR-Rotation durchgeführt (zig-zag bzw. zag-zig):



Beispiel Top-Down Splay

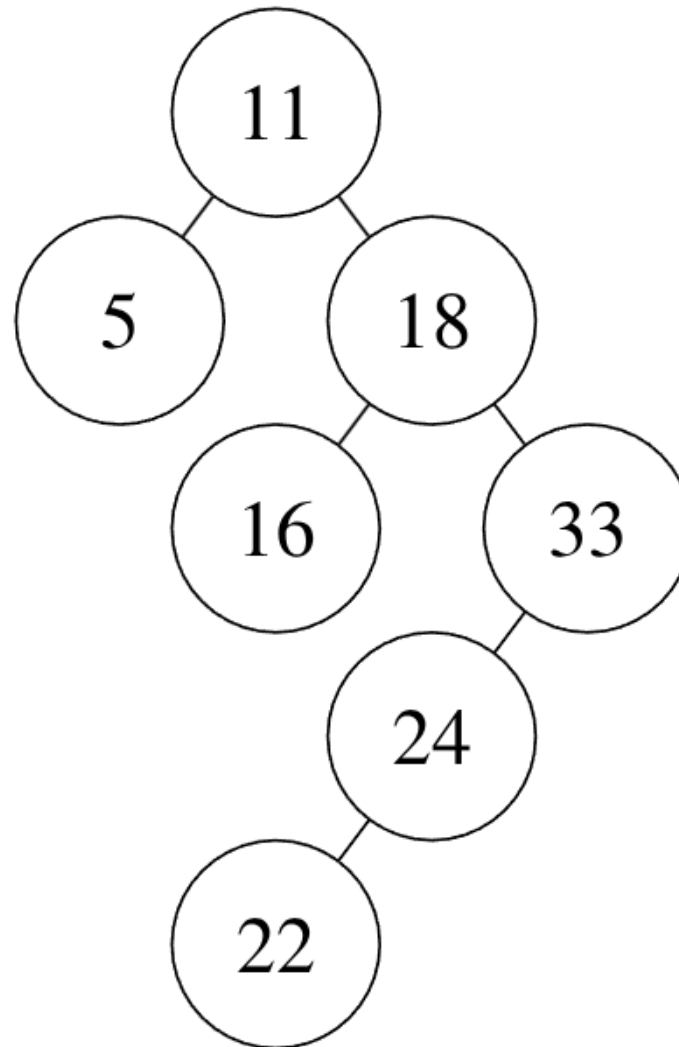
- falls der gesuchte Knoten x und sein Vorgänger p beide linke bzw. rechte Unterbäume sind werden zwei Einfachrotationen in der Top-Down-Reihenfolge durchgeführt (zig-zig bzw. zag-zag):



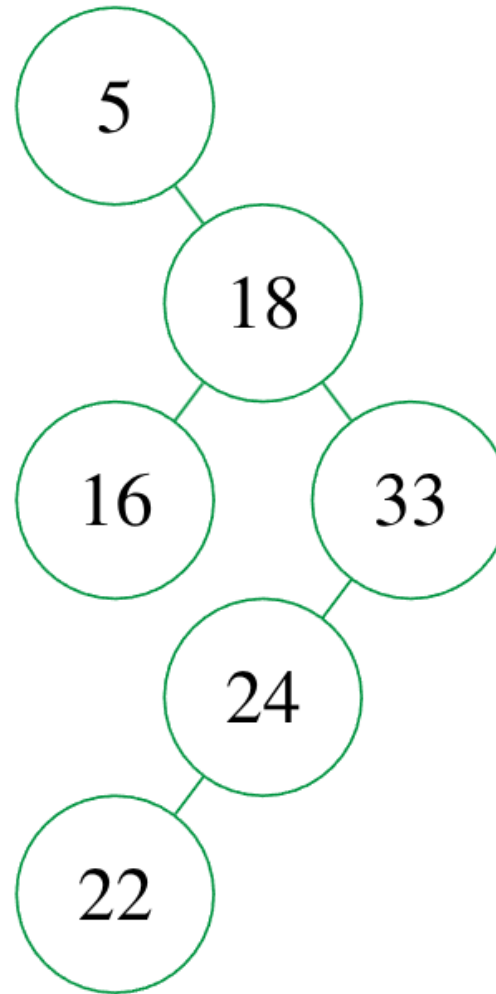
Aufgabe 3

3 a)

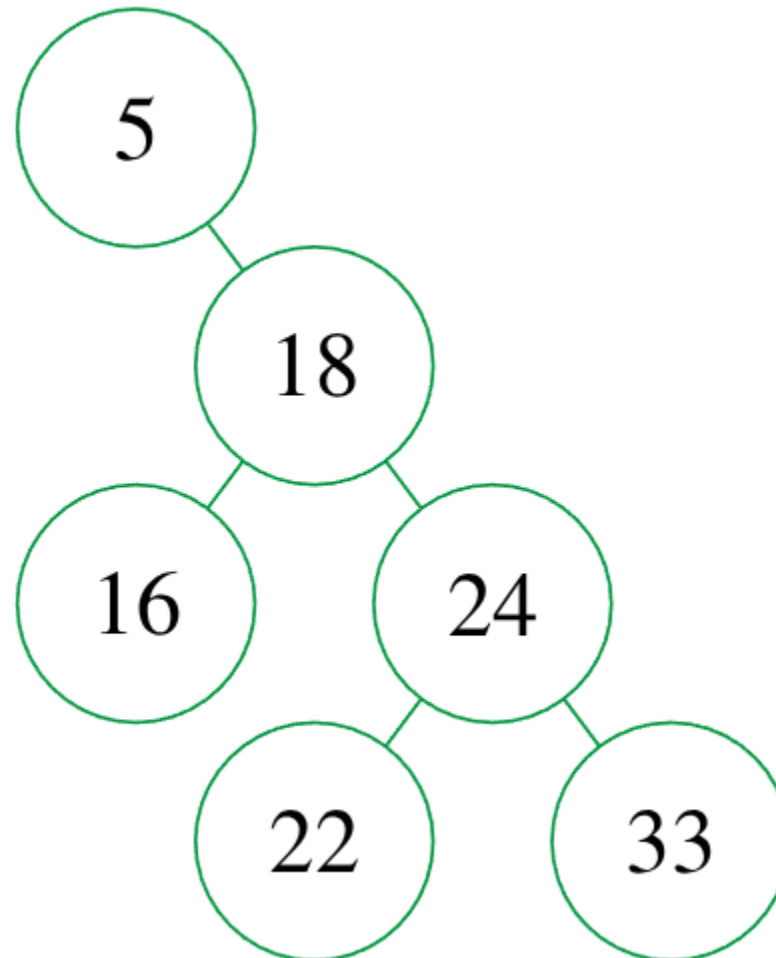
Gegeben: Bottom-Up Splay Tree



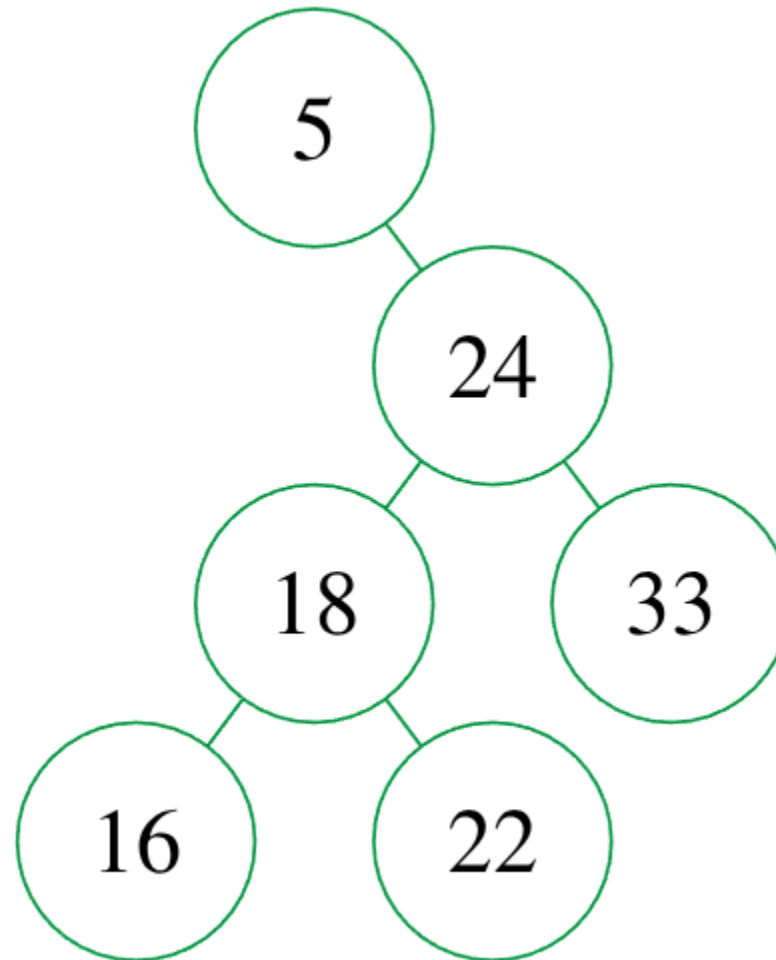
- Schlüssel 11 löschen



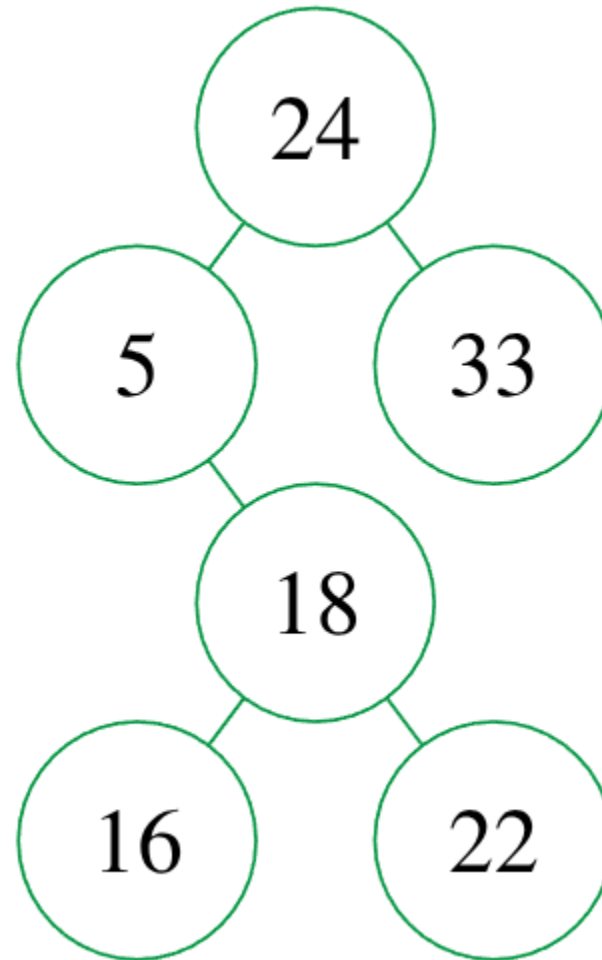
- Schlüssel 24 suchen
- Rechtsrotation mit 24 und 33



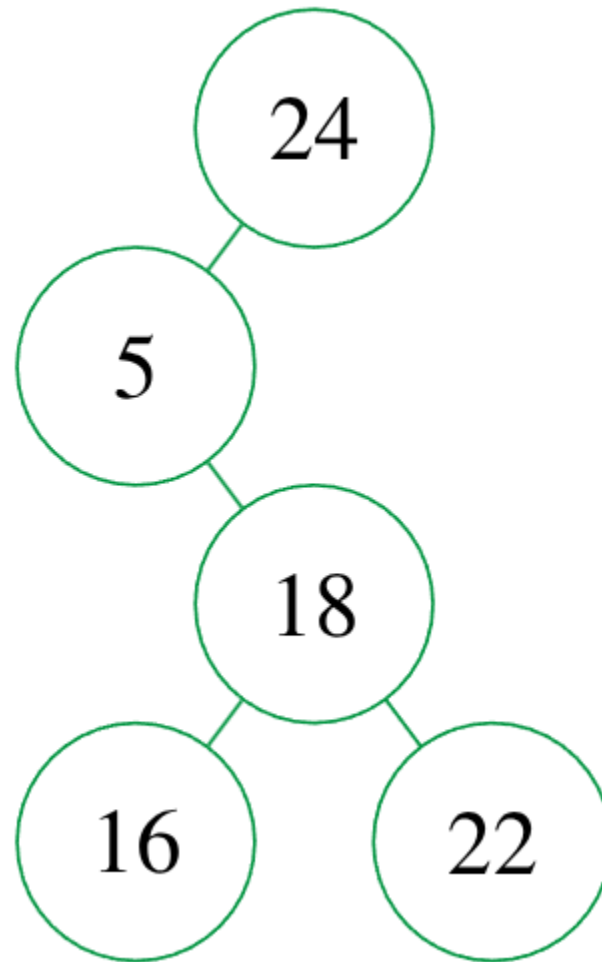
- Linksrotation mit 24 und 18



- Linksrotation mit 24 und 5



- Schlüssel 33 löschen



3 b)

```
1 public static int [] sort(int[] input) {
2     SplayTree<Integer> tree = nes SplayTree<>();
3     for (int i = 0; i < input.length; i++) {
4         tree.add(input[i]);
5     }
6     int[] output = new int[input.length];
7     for (int i = 0; i < input.length; i++) {
8         int min = tree.min();
9         tree.remove(min);
10        output[i] = min;
11    }
12    return output;
13 }
```

- die Funktion `min()` (Zeile 8) gibt den kleinsten Schlüssel aus
 - es kann davon ausgegangen werden, dass die amortisierte Laufzeit der `min()`-Funktion genauso schnell ist, wie die Suche eines Elements in einem Splay-Baum

Amortisierte Laufzeit

Beispiel: Dynamisches Array

- ein Algorithmus speichert Daten in einem Array von fester Länge
- wenn das Array voll ist, erstellt der Algorithmus ein neues Array mit doppelter Länge, kopiert alle Werte aus dem alten Array und fügt das neue Element hinzu
- normalerweise hat die Funktion “Element hinzufügen” eine Laufzeit von $O(1)$
 - aber dann, wenn das Array voll ist, hat die gleiche Operation eine Laufzeit von $O(n)$
- das heißt, eigentlich müsste die Funktion “Element hinzufügen” eine Laufzeit von $O(n)$ haben, aber für die meiste Zeit ist das nicht der Fall
- die amortisierte Laufzeit bietet eine bessere Darstellung der tatsächlichen Laufzeit

Definition

Die amortisierte Laufzeit ist der Durchschnittswert pro Operation, wenn eine Folge von n Operationen betrachtet wird.

- eine einzelne Operation hat vielleicht eine schlechtere Laufzeit
- aber die meisten Operationen haben eine bessere Laufzeit
- die amortisierte Laufzeit “verteilt” die schlechte Laufzeit der einen Operation auf die besseren Laufzeiten der anderen Operationen

Für mehr Infos:

- <https://www.ai-futureschool.com/de/informatik/informatik-analyse-der-amortisierten-algorithmen-effizienz-2024.php>
- https://ac.informatik.uni-freiburg.de/lak_teaching/ws04_05/algotheo/Misc/Slides/06_Amortisierte_Analyse.pdf

```
1 public static int [] sort(int[] input) {  
2     SplayTree<Integer> tree = nes SplayTree<>();  
3     for (int i = 0; i < input.length; i++) {  
4         tree.add(input[i]);  
5     }  
6     int[] output = new int[input.length];  
7     for (int i = 0; i < input.length; i++) {  
8         int min = tree.min();  
9         tree.remove(min);  
10        output[i] = min;  
11    }  
12    return output;  
13 }
```

Was ist die Laufzeit bei einem Worst-Case Szenario und warum?

- Lösung: $O(n \cdot \log(n))$
- Begründung:
 - die Laufzeit für die Operationen Suchen, Einfügen, Löschen und Finden des Minimum sind jeweils $O(\log(n))$
 - in beiden for-Schleifen werden diese Operationen n mal durchgeführt ($n = \text{input.length}$)
 - das heißt, selbst wenn eine Operation mal länger braucht, liegt die amortisierte Laufzeit bei $O(n \cdot \log(n))$

Wiederholung AVL- Bäume

AVL-Baum

- ein AVL-Baum ist ein binärer Suchbaum, bei dem sich die Höhe h der Teilbäume jedes Knoten um maximal eine Ebene unterscheiden
 - nachdem Elemente hinzugefügt oder entfernt wurden, kann es sein, dass der Baum neu balanciert werden muss, damit die Bedingung noch gilt
 - ob ein Baum rebalanciert werden muss, zeigt die Balance b an, die bei jedem Knoten steht
 - die Balance darf entweder nur -1 , 1 oder 0 sein, ansonsten muss der Baum rebalanciert werden

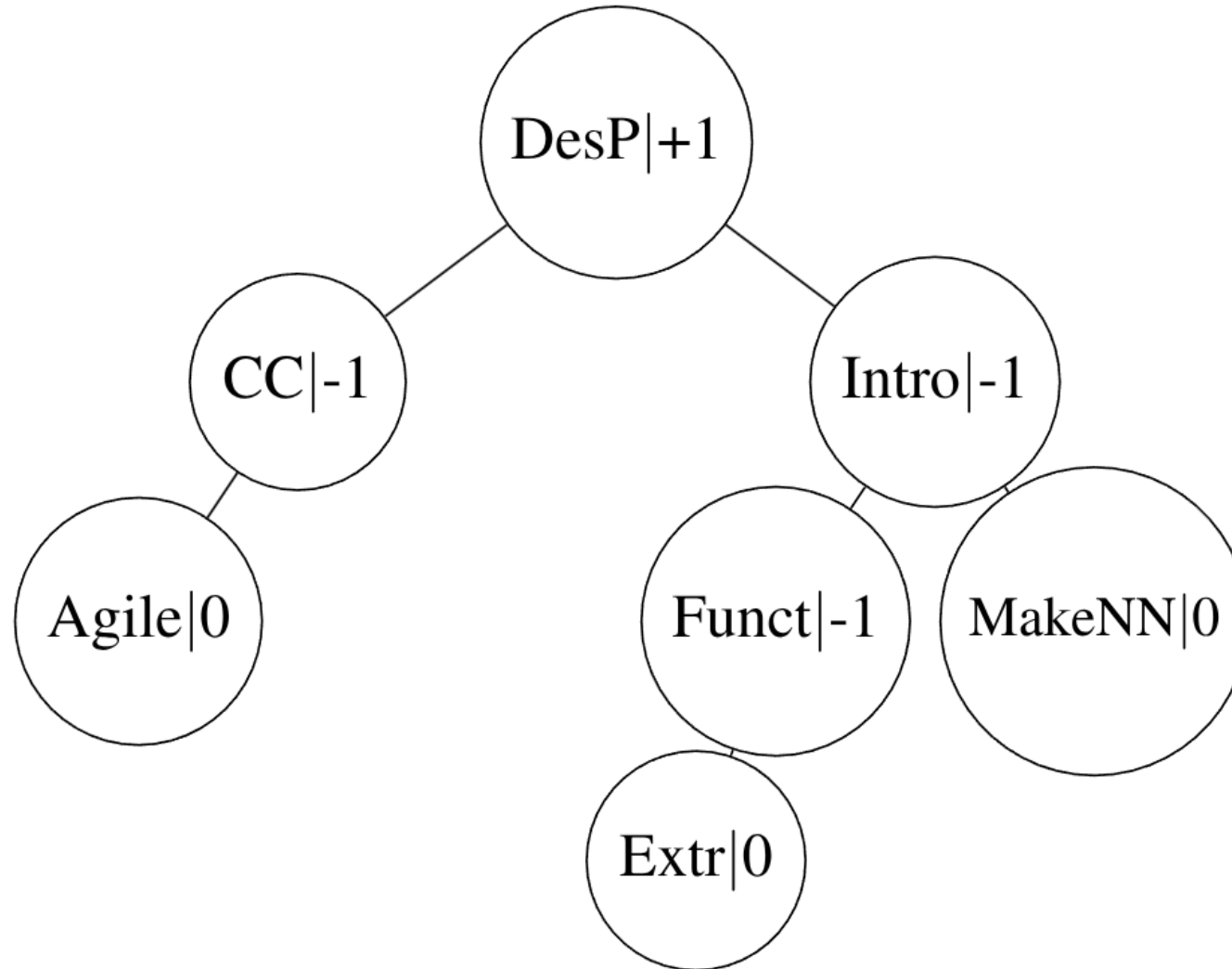
$$b = h(\textit{rechter Teilbaum}) - h(\textit{linker Teilbaum})$$

Aufgabe 4

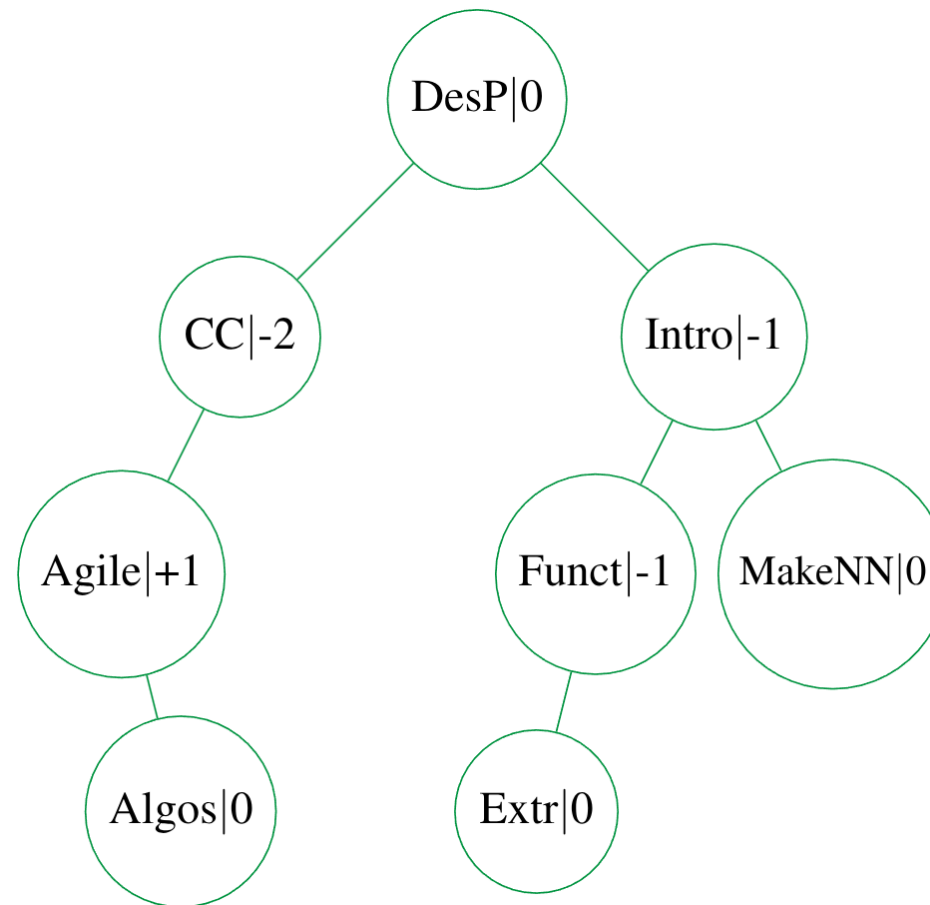
Nummer	Titel	Autor	Jahr	Kürzel
1	Design Patterns	E. Gamma, et al.	1994	DesP
2	Clean Code	Robert C. Martin	2008	CC
3	Make Your Own Neural Network	Tariq Rashid	2016	MakeNN
4	Agile Software Development	Robert C. Martin	2002	Agile
5	Introduction to Algorithms	T. H. Cormen, et al.	1989	Intro
6	Functional Thinking: Paradigm Over Syntax	Neal Ford	2014	Funct
7	Extreme Programming Explained: Embrace Change	Kent Beck	1999	Extr
8	Algorithms for Reinforcement Learning	Csaba Szepesvari	2010	Algos
9	The Software Craftsman	Robert C. Martin	2014	TheSW
10	Test Driven Development: By Example	Kent Beck	2002	TestDD
11	Programming Pearls	Jon Bentley	1986	PP
12	Building Your Own Compiler with C++	Jim Holmes	1994	Build

4 a)

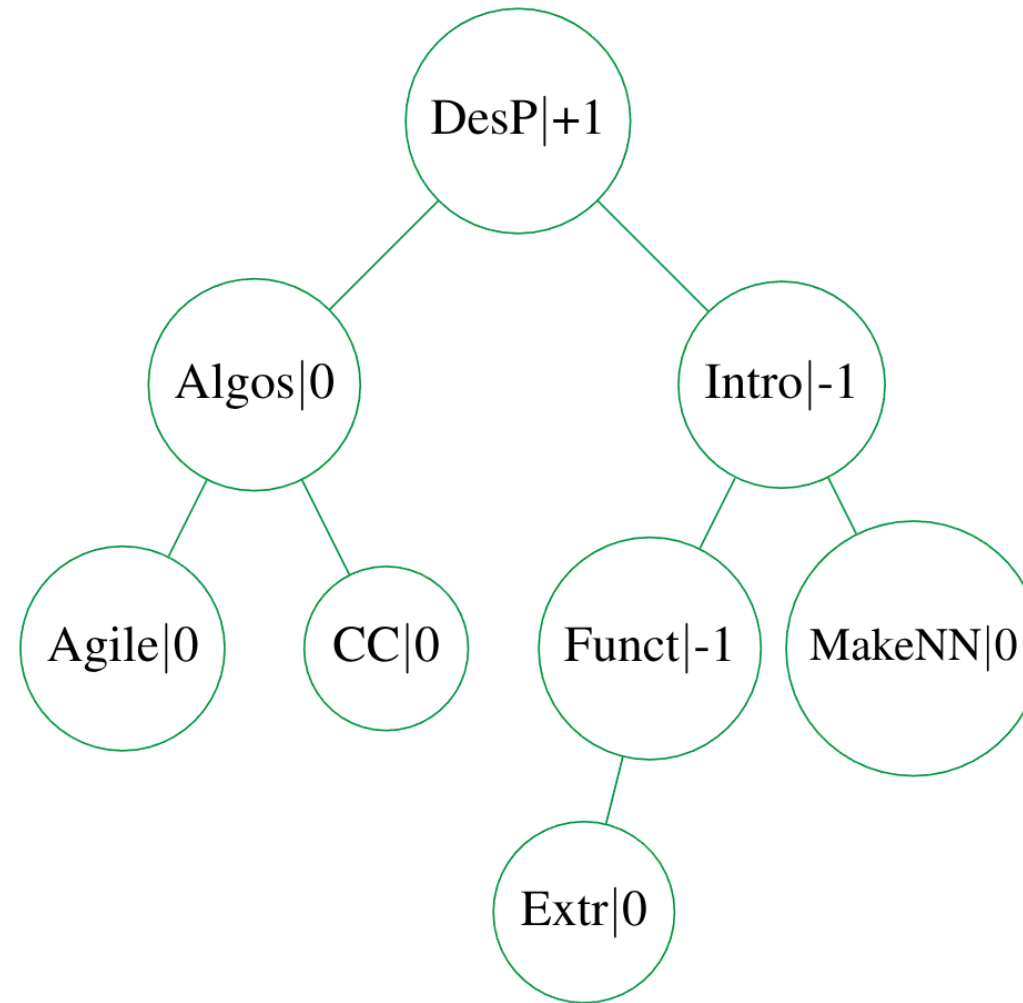
Gegeben:



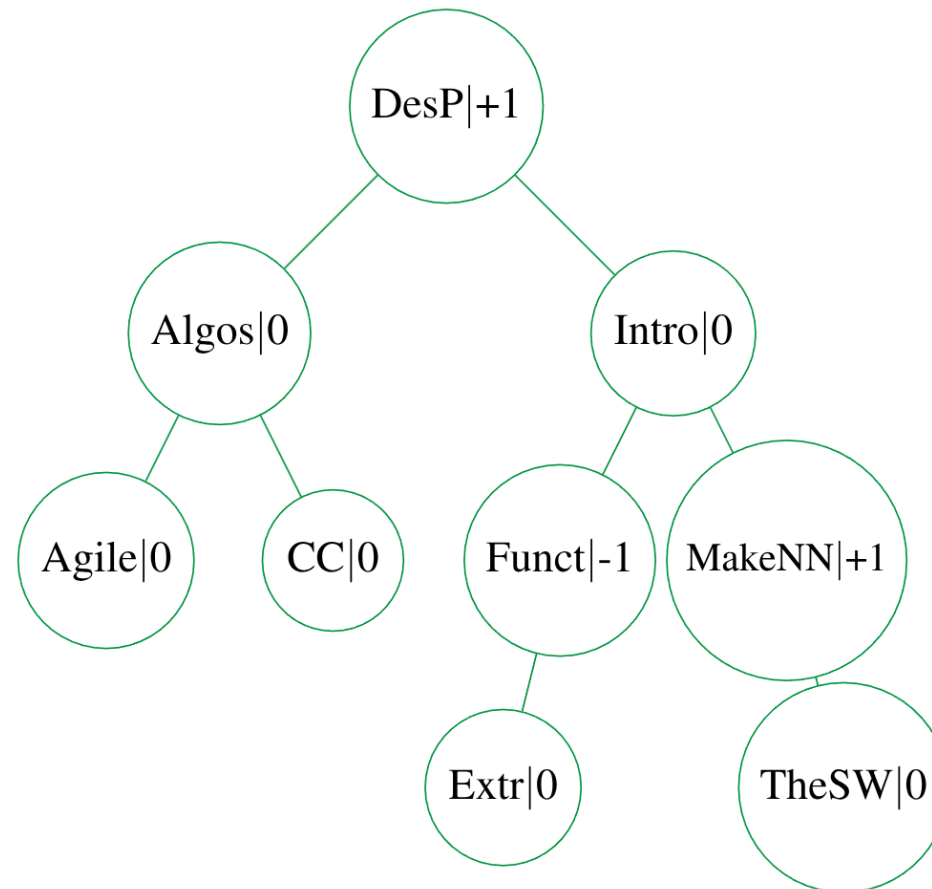
- Einfügen: 8 Algos
 - liegt links von CC und rechts von Agile



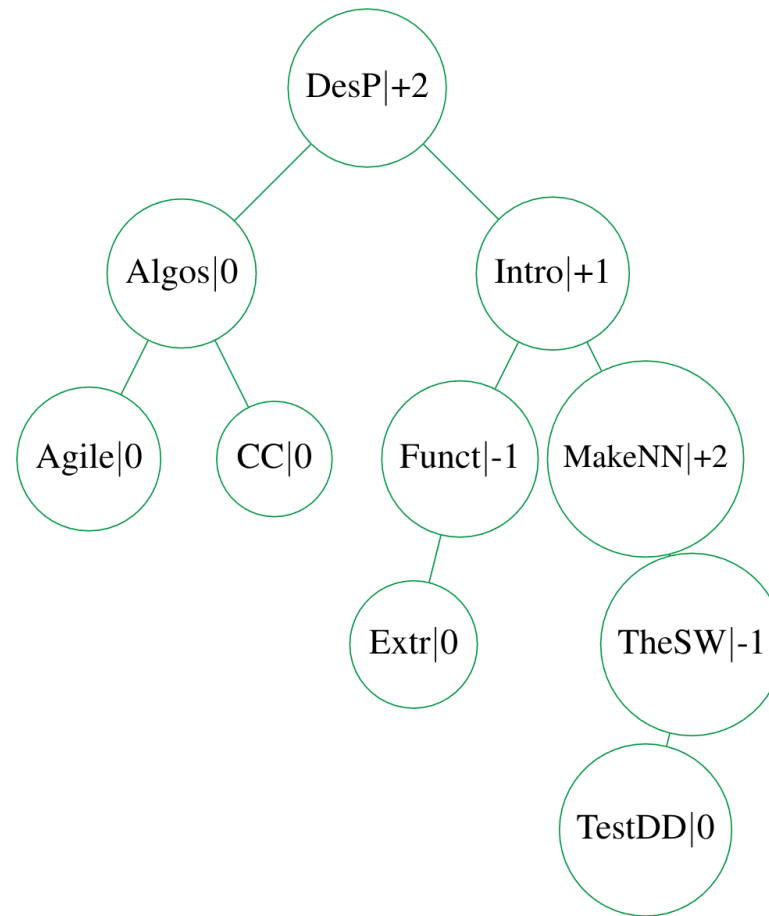
- $b = -2$ bei CC, daher LR-Rotation:



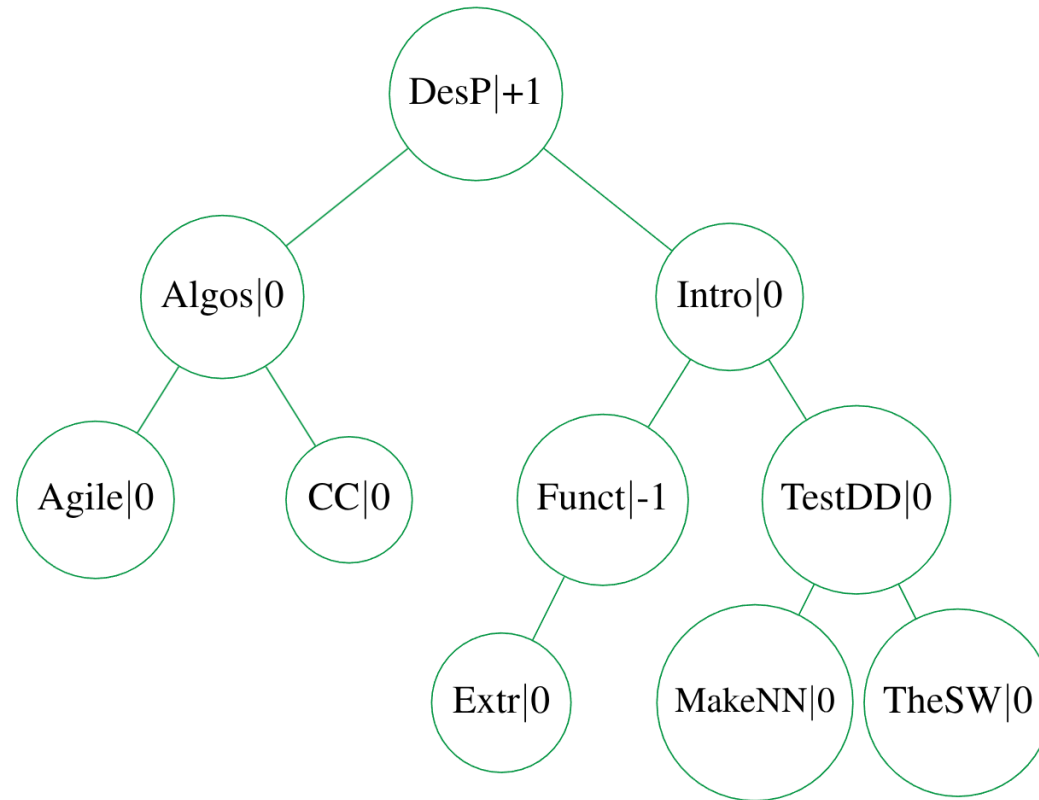
- Einfügen: 9 TheSW
 - liegt rechts von MakeNN



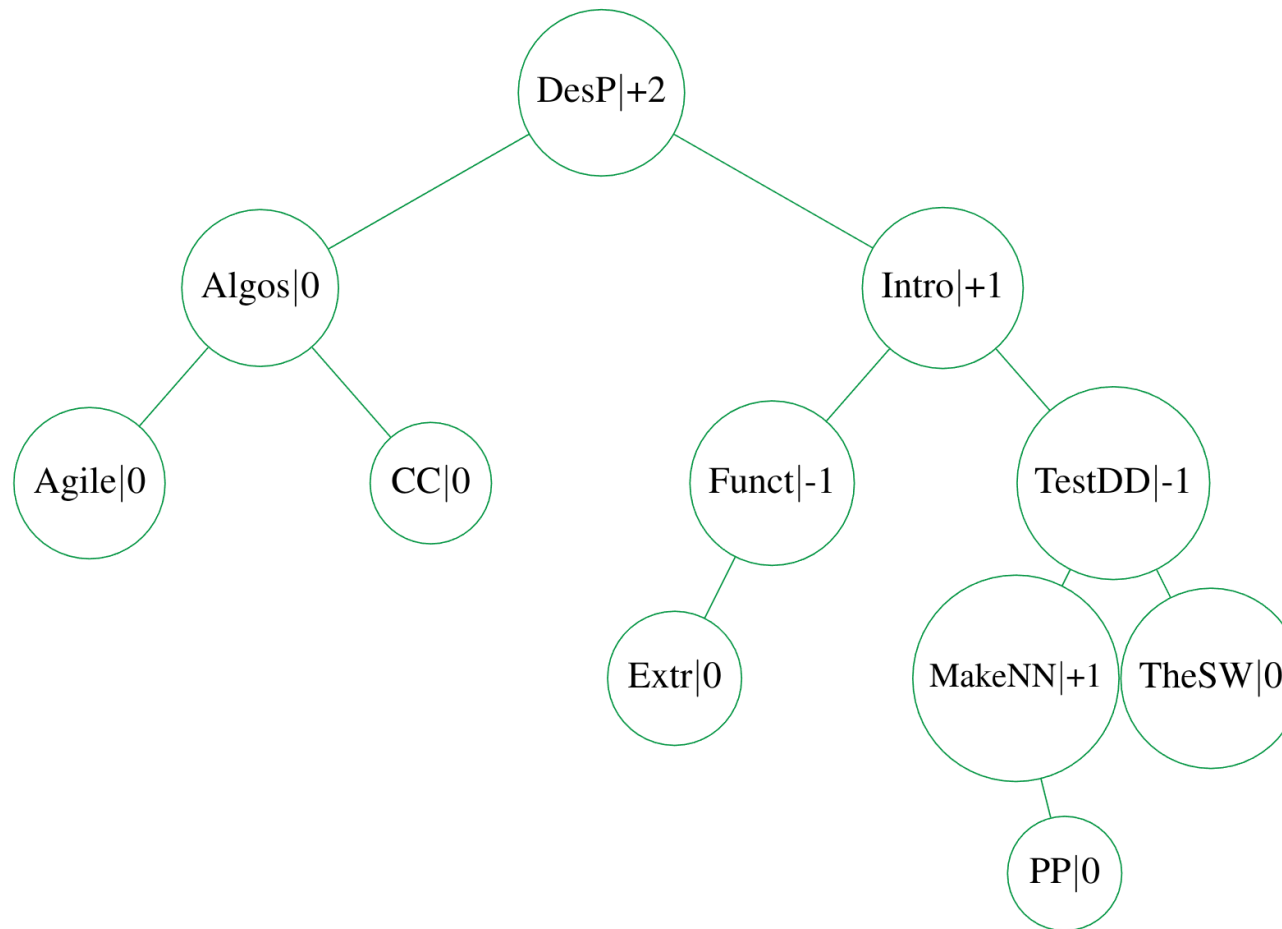
- Einfügen: 10 TestDD
 - liegt rechts von MakeNN und links von TheSW



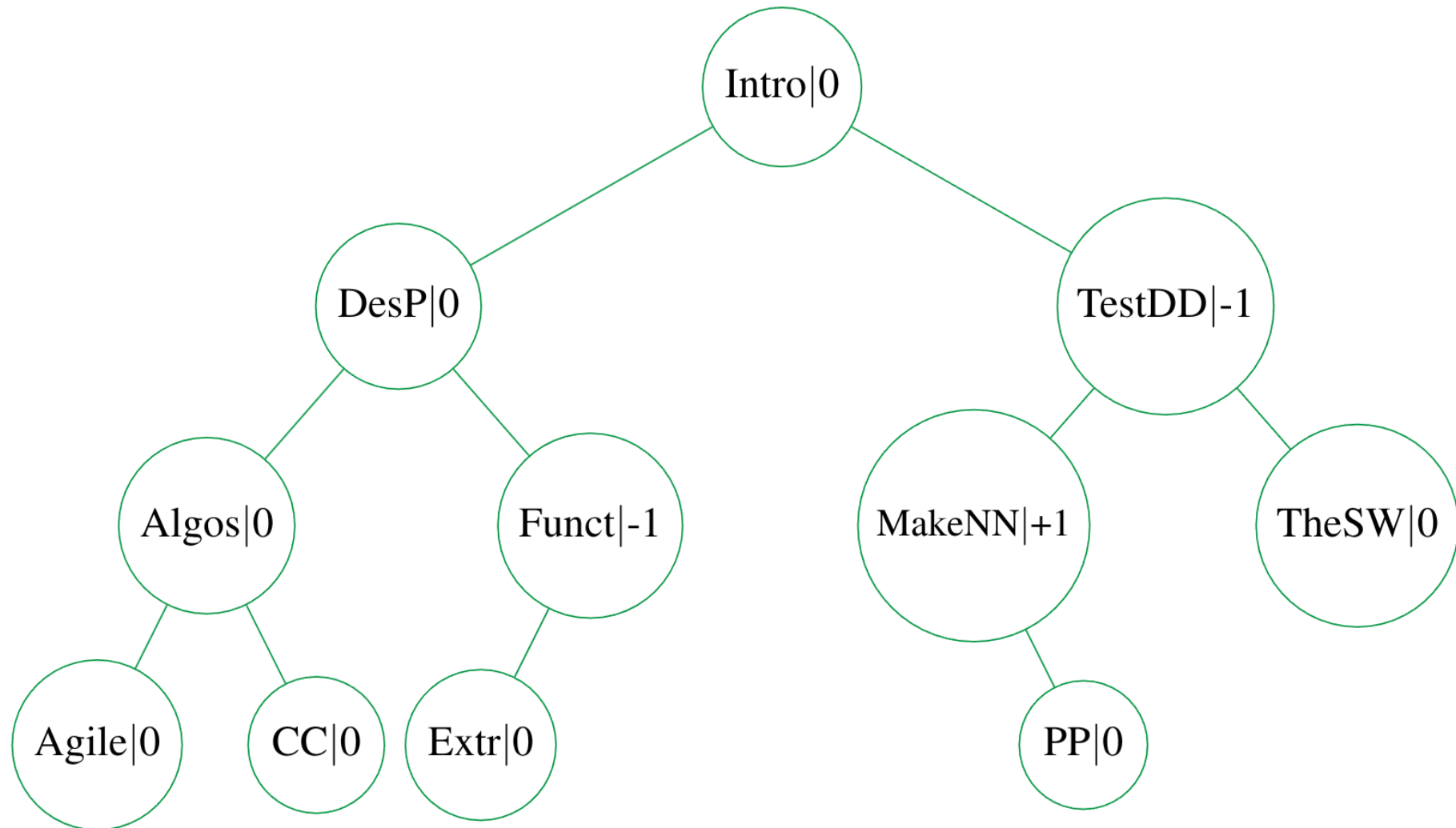
- $b = +2$ bei MakeNN, daher RL-Rotation



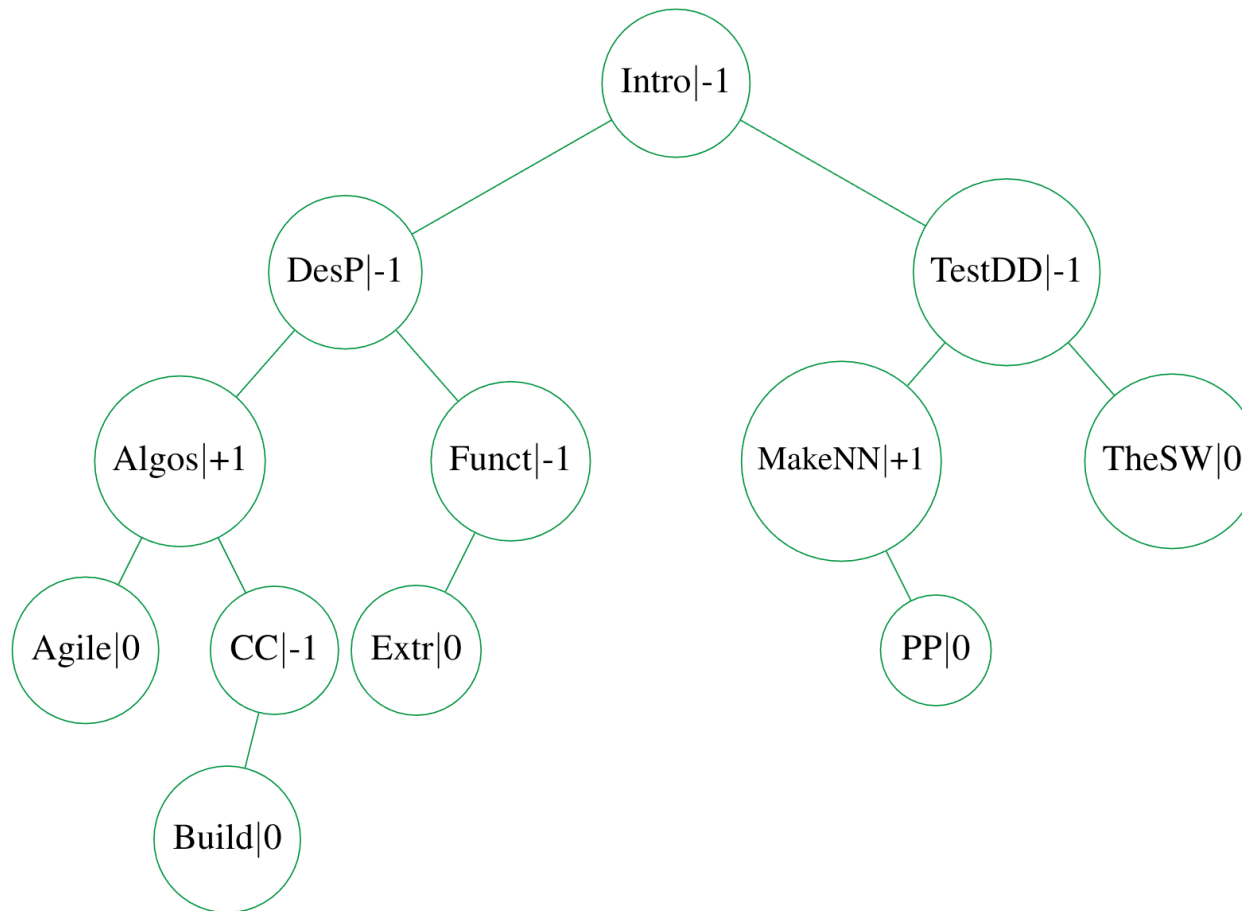
- Einfügen: 11 PP
 - liegt links von TestDD und rechts von MakeNN



- $b = +2$ bei DesP, daher Links-Rotation

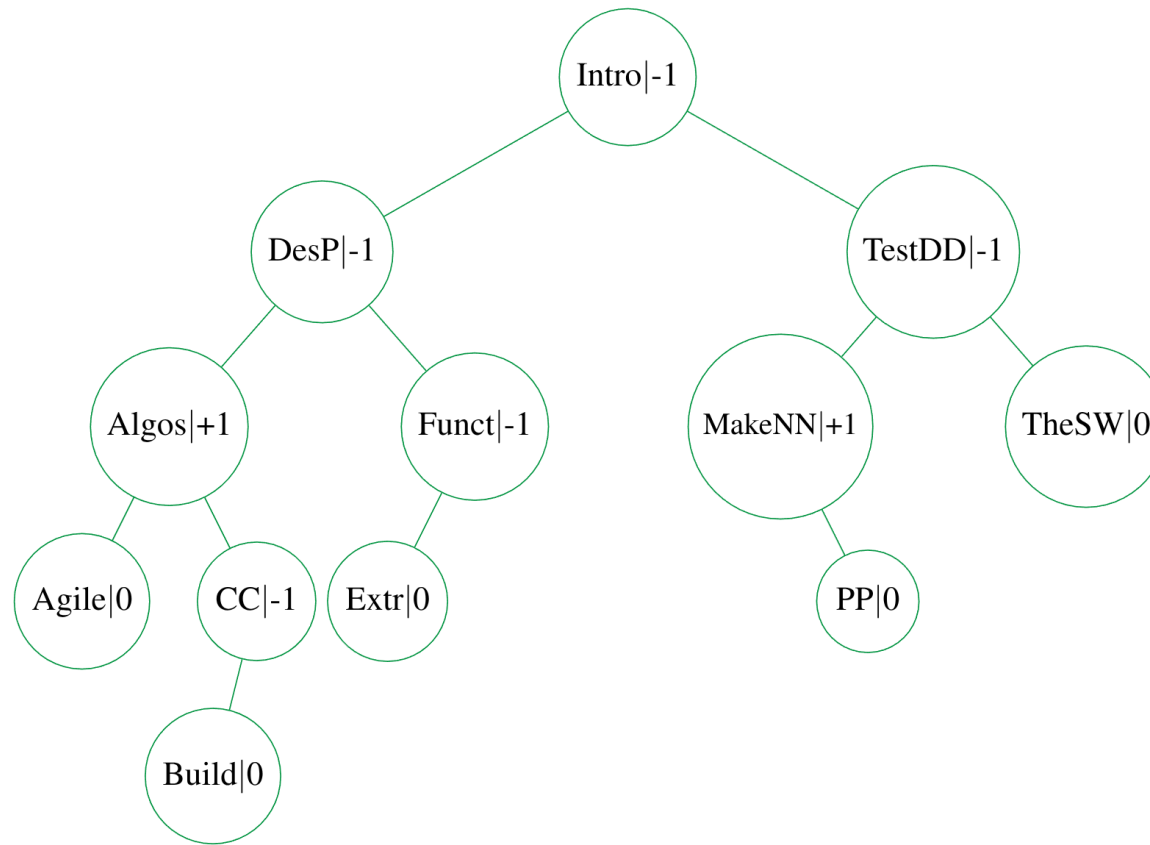


- Einfügen: 12 Build
 - liegt zwischen Algos und CC



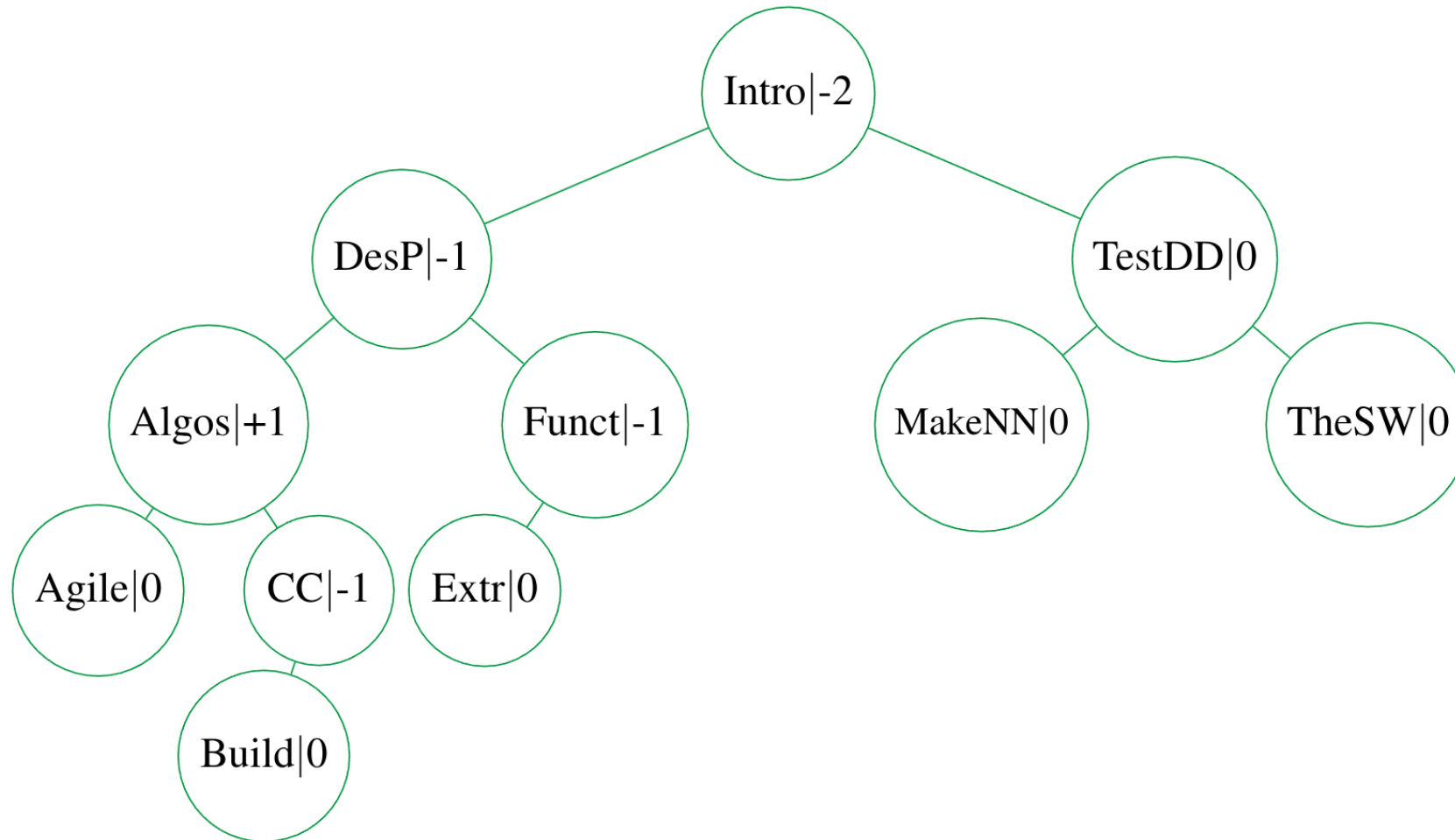
4 b)

Entferne die drei ältesten Bücher ausgehend vom Erscheinungsjahr.

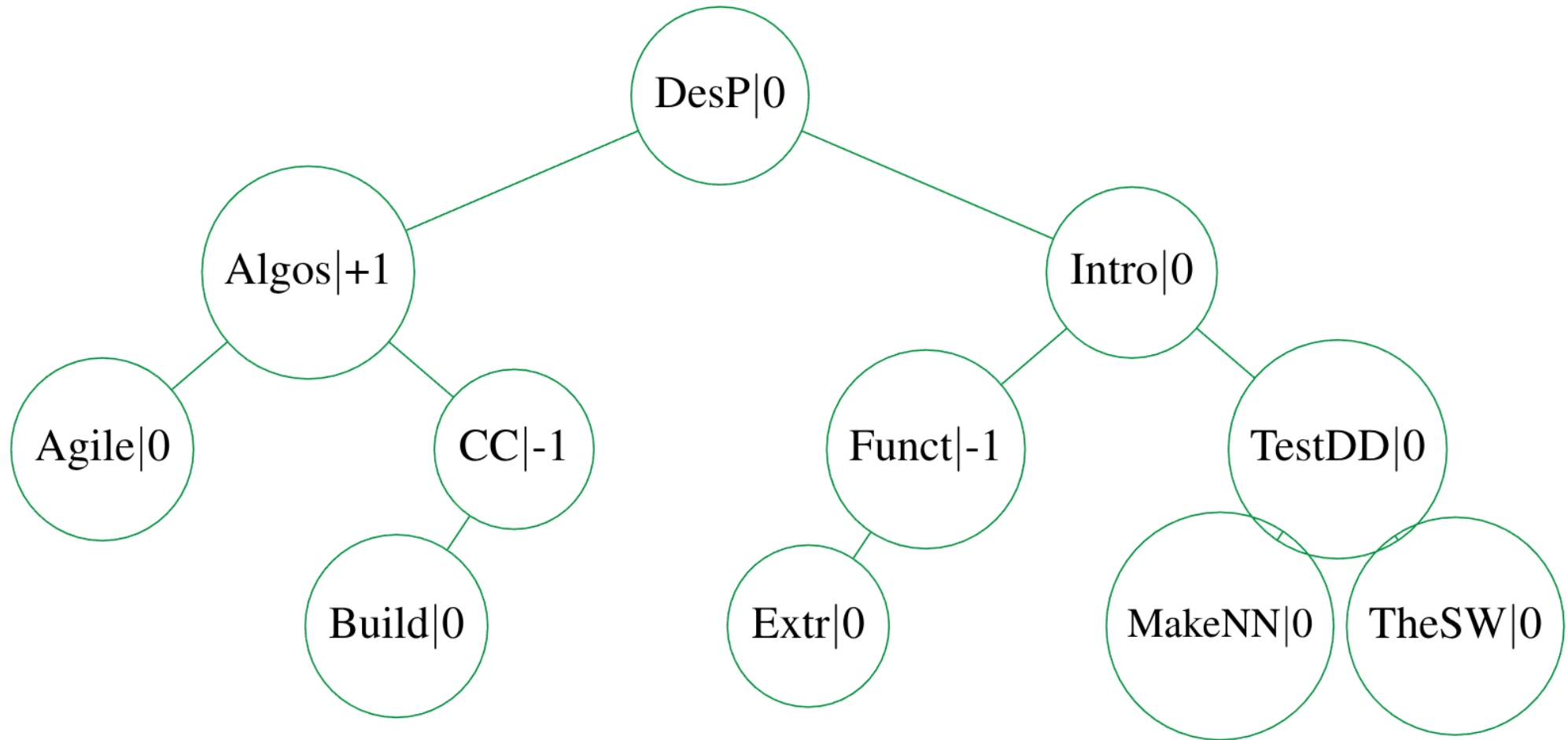


- PP (1986)
- Intro (1989)
- Build (1994)

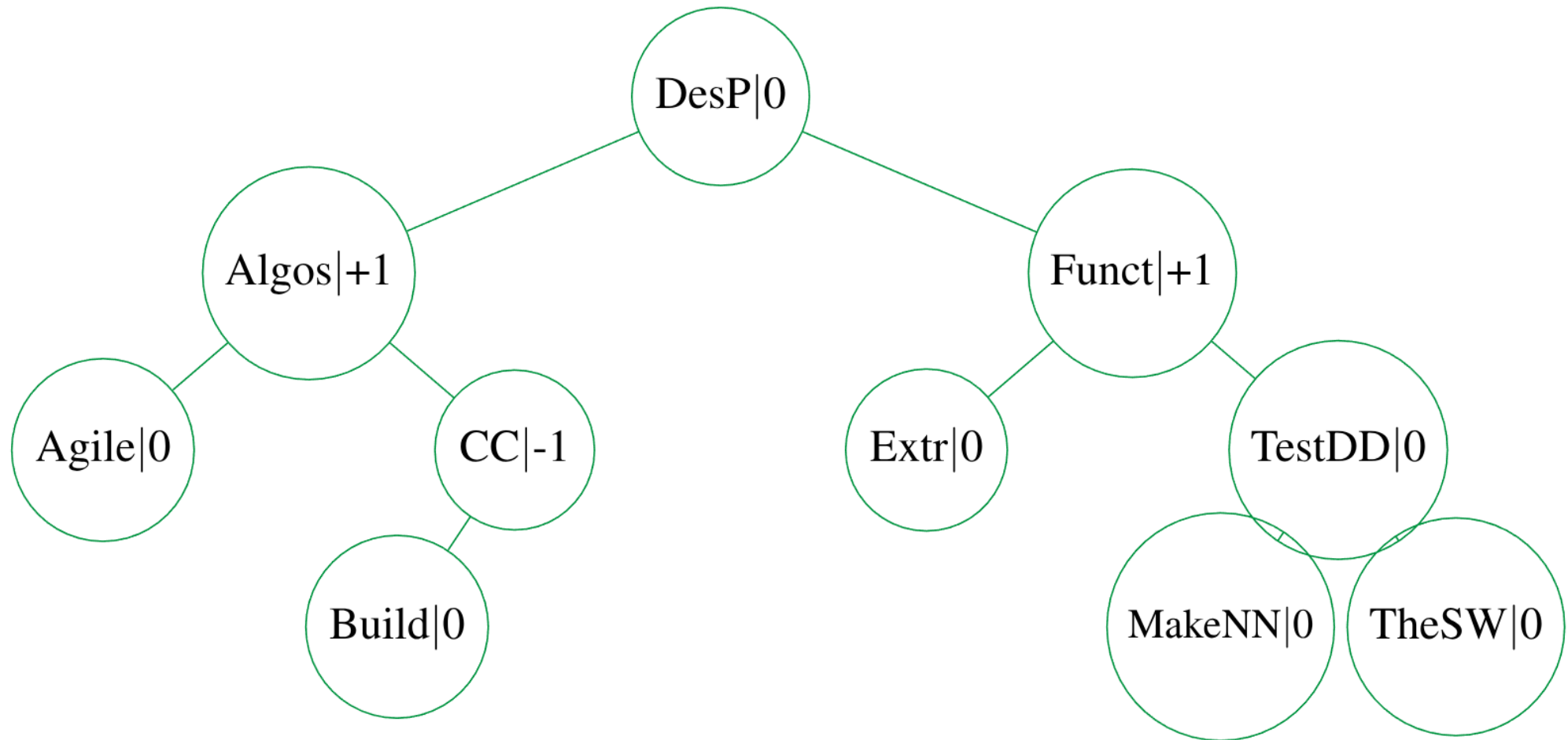
- Entferne: PP (1986)



- $b = -2$ bei Intro, daher Rechts-Rotation



- Entferne: Intro (1989)



- Entferne: Build (1994)

